

Advance Java LAB



Rajasthan Technical University, Kota (Raj.)

Software Requirements Specification(SRS)

Project Name-
JavaLabPortal

Submitted By:

- | | |
|---------------------|------------|
| 1. Krishna Gupta | 24EARAD083 |
| 2. Karan Ramlakhani | 24EARAD076 |
| 3. Kanishq Chasta | 24EARAD074 |
| 4. Jayesh Sharma | 24EARAD070 |
| 5. Juned Hussain | 24EARAD072 |

Under the supervision of

Er. Ram Babu Buri
Assistant Professor



Session: 2025-26

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

TABLE OF CONTENTS

Section Heading	Core Topics Covered
1. Introduction	Purpose of the System, Project Scope, Objectives, Document Conventions, Intended Audience, References
2. Overall Description	Product Perspective, Product Features, User Classes, Operating Environment, Design Constraints, Assumptions
3. Internal Interface Requirements	Input Interface, Processing Interface, Output Interface, Integration Interface, Non-Functional Internal Requirements
4. External Interface Requirements	User Interface, Software Interface, Hardware Interface, Communication Interface
5. Other Functional Requirements	Traceable Functional Requirements (FR-01 to FR-10) with Descriptions
6. Other Non-Functional Requirements	Quality Metrics (Performance, Reliability, Scalability, Usability, Maintainability, Security)
7. Analysis Models	Use Case Model, Class / Module Architecture Diagram, Activity / Practical Workflows
8. Other Requirements	Data Residency, Privacy, Accessibility, Deployment, Backup, Logging, Dependencies, Training

1. Introduction

1.1 Purpose of the System

The purpose of the JavaLabPortal system is to provide a centralized, web-based practical learning and examination management platform for engineering students, faculty instructors, and laboratory coordinators. It eliminates disconnected desktop workflows in academic programming laboratories by centralizing curriculum experiment delivery, theory documentation access, student performance records, and dynamic computational utilities.

The platform is engineered using robust enterprise Java technologies, leveraging Java Servlets (Java EE 8), JavaServer Pages (JSP), and MySQL 8.0, structured strictly under the Model-View-Controller (MVC) architectural pattern. JavaLabPortal focuses on high reliability, clear role segregation, relational data integrity, and seamless accessibility across academic labs.

1.2 Project Scope

JavaLabPortal is designed specifically for academic institutions affiliated with Rajasthan Technical University (RTU), Kota, supporting the Advance Java Lab curriculum (5th/6th Semester B.Tech in Computer Science and Artificial Intelligence & Data Science).

The system will:

- Provide role-based authentication and secure session management for Students and Faculty Instructors.
- Publish and organize dynamic laboratory experiments with detailed problem statements, objectives, and code structures.
- Deliver structured theory manuals and conceptual documentation directly linked to each syllabus experiment.
- Maintain comprehensive student lifecycle records, class section assignments, and academic enrollments.
- Provide interactive Java computation tools (MVC Expression Calculator and Arithmetic Engine) for practical demonstration.
- Enable students to submit qualitative lab feedback and track administrative review scores.
- Operate reliably in both local network laboratory setups and containerized cloud environments (Apache Tomcat 9 on Railway Cloud).

The system will not:

- Support arbitrary unmonitored shell access or execution of hostile native binaries outside the sandbox JVM.
- Provide unstructured real-time peer-to-peer social messaging or unrelated video broadcasting.
- Act as a general-purpose public file-sharing drive; storage is strictly scoped to academic lab entities.

1.3 Objectives

- Digitize and streamline academic practical laboratory workflows for students and educators.
- Ensure clean architectural separation of concerns through an enterprise Model-View-Controller (MVC) design.
- Enforce relational data integrity across students, sections, experiments, theory content, and faculty records.
- Deliver sub-second response times for laboratory content retrieval and computational operations.
- Maintain high portability, enabling effortless deployment on standard Apache Tomcat 9 containers and cloud environments.

1.4 Document Conventions

- FR-XX: Represents Functional Requirements defining distinct system behaviors and capabilities.
- NFR-XX: Represents Non-Functional Requirements defining performance, security, and quality benchmarks.
- UML Notation: Diagrams conform to standard Unified Modeling Language (UML 2.5) specifications.
- Java EE Conventions: Code elements, servlet endpoints, and DAO classes adhere to standard Java naming conventions (PascalCase for classes, camelCase for methods/variables, kebab-case for URL mappings).

1.5 Intended Audience and Reading Suggestions

This specification is written for evaluators, software engineering leads, academic faculty, and system administrators. The table below outlines reading recommendations based on stakeholder roles:

Target Audience Role	Reading Focus & Recommended Sections
Laboratory Instructors / Evaluators	Review Functional Requirements (Section 5) and Use Case Models (Section 7) to verify academic alignment.
Software Developers & Maintainers	Examine Product Architecture (Section 2), Internal Interfaces (Section 3), and Class Models (Section 7).
System & Lab Administrators	Refer to Operating Environment (Section 2.4), External Interfaces (Section 4), and Deployment (Section 8).
Quality Assurance / Testers	Utilize Traceable Functional Requirements (Section 5) and Non-Functional Requirements (Section 6) to formulate test plans.

1.6 References

- Oracle Java Enterprise Edition (Java EE 8) Servlet 4.0 Specification (JSR 369).
- Apache Tomcat 9.0 Architecture & Configuration Guide (tomcat.apache.org).
- MySQL 8.0 Reference Manual & JDBC Connector/J 8.0 Driver Documentation.
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Rajasthan Technical University (RTU), Kota: B.Tech Scheme & Syllabus for Advance Java Laboratory (AI & DS / CSE).

2. Overall Description

2.1 Product Perspective

JavaLabPortal is a standalone, client-server web application architected under the Model-View-Controller (MVC) paradigm. It decouples presentation markup, computational servlet routing, and relational persistence into distinct, manageable tiers:

- Presentation Tier (Frontend): Built with semantic HTML5, CSS3, modern JavaScript, and dynamic JavaServer Pages (JSP). Interfaces are served from the web application context root.
- Application Tier (Controllers & Services): Java Servlets compiled against Java 11 JDK, managing authentication sessions, validation logic, computational processing, and HTTP dispatching.
- Persistence Tier (Database): MySQL 8.0 relational database executing normalized SQL schemas, accessed exclusively via clean JDBC Data Access Objects (DAOs) and parameterized queries.
- Deployment Container: Standardized WAR archive packaged and executed on Apache Tomcat 9.0, running locally or inside Railway Cloud Docker environments.

Key System Subsystems:

- **Authentication & Session Subsystem:** Handles credential verification for students and teachers, generating secure HTTP sessions with inactivity timeouts.
- **Academic Entity Management Subsystem:** Coordinates CRUD operations for students, classes, and lab sections with foreign key validation.
- **Experiment & Theory Subsystem:** Delivers lab exercises, code templates, syllabus objectives, and theory documentation.
- **Interactive Computation Subsystem:** Executes dynamic mathematical algorithms via CalculatorServlet and ArithmeticServlet.
- **Feedback & Audit Subsystem:** Collects student evaluations and records portal operational logs.

2.2 Product Features

- **Role-Based User Authentication:** Distinct access workflows for Faculty Instructors and Enrolled Students.
- **Centralized Experiment Catalog:** Comprehensive listing of RTU Advance Java experiments with dynamic detail views.
- **Integrated Theory Repository:** Rich documentation explaining core concepts (JDBC, Servlets, JSP, Multithreading, Sockets).
- **Student & Class Management:** Faculty capabilities to register students, assign sections, and update enrollment records.
- **Live Interactive MVC Utilities:** Built-in Calculator and Arithmetic engines demonstrating real-time Java EE computation.
- **Laboratory Review System:** Interactive feedback submission module recording ratings and qualitative reviews.
- **Cloud-Native Readiness:** Dynamic database environment detection supporting both local MySQL and cloud MySQL.

2.3 User Classes and Characteristics

User Class	Role Description & Academic Responsibilities	Technical Skill
Students	Enrolled engineering candidates who view lab experiments, read theory manuals, test interactive tools, and submit feedback.	Basic (Browser navigation)
Faculty / Lab Instructors	Academic professors who manage lab experiments, enroll students, organize class sections, and oversee reviews.	Intermediate (Academic Portal Management)
System Administrator	Lab technicians and DevOps leads managing database backups, Tomcat server uptime, and cloud deployment pipelines.	Advanced (Server & DB Administration)
Developer / Contributor	Student engineers maintaining the MVC codebase, adding new servlet controllers, and upgrading schemas.	Advanced (Java EE & SQL Engineering)

2.4 Operating Environment

- Client Environment: Any modern web browser (Google Chrome 110+, Mozilla Firefox 110+, Microsoft Edge, Safari) on Windows, Linux, macOS, or mobile devices.
- Application Server: Apache Tomcat 9.0 (Servlet Specification 4.0, JSP 2.3 support).
- Java Runtime: Java Development Kit (JDK) 11 LTS or Java SE 8.
- Database Engine: MySQL Community Server 8.0 or cloud-hosted MySQL on Railway Cloud / AWS RDS.
- Operating System: Linux (Ubuntu 22.04 LTS container in production) or Windows 10/11 for local development in Eclipse IDE.

2.5 Design and Implementation Constraints

- Architecture Constraint: Strict MVC architecture—JSPs must never contain raw business logic or direct JDBC database statements.
- Database Integrity: Relational foreign key constraints must be enforced across students, class sections, and feedback records.
- Security Constraint: Passwords and sensitive parameters must be transmitted via HTTP POST and validated on the server side.
- Zero External Heavy Frameworks: Developed using standard Java EE APIs without monolithic external frameworks, adhering strictly to university practical exam evaluation rubrics.

2.6 User Documentation

- README.md: Detailed repository documentation covering local Eclipse IDE configuration, MySQL schema import, and cloud deployment.
- Database Schema Guide: SQL script (database.sql) documenting all 6 tables, primary/foreign keys, and pre-seeded RTU curriculum data.
- Lab Manual & Code Walkthrough: In-app theory content explaining Java EE architecture, servlet life cycles, and database connectivity.

2.7 Assumptions and Dependencies

- The client workstation has an active network connection to the host Tomcat server (LAN port 8080 or public HTTPS domain).
- The MySQL database server is running, accepting connections, and has created the javalabdb schema.
- The client browser has JavaScript enabled for dynamic form validation and asynchronous UI enhancements.

3. Internal Interface Requirements

3.1 Input Interface

The JavaLabPortal backend accepts structured inputs submitted via standard HTTP protocol mechanisms:

- Request Methods: HTTP GET for resource retrieval and search queries; HTTP POST for credential submission, entity modification, and computational execution.
- Encoding Types: application/x-www-form-urlencoded for standard forms; URL query string parameters for action routing (?action=list, ?action=delete, ?id=1).
- Input Sanitization: All text inputs undergo parameter null-checks, whitespace trimming, and type casting prior to processing.

3.2 Processing Interface

Internal application processing is coordinated through dedicated Java EE controllers:

- Authentication Processing: LoginServlet inspects role parameters ('student' vs 'teacher') and queries the respective table using PreparedStatement parameters.
- Data Management Processing: StudentServlet and ClassServlet intercept CRUD actions, invoke respective DAO methods, and manage database transactions.
- Computational Processing: CalculatorServlet parses numerical tokens and mathematical operators, executing evaluation within a controlled try-catch block.
- Resource Management Processing: ExperimentServlet and TheoryServlet dynamically query curriculum catalogs and bind result objects into request attributes.

3.3 Output Interface

The system produces dynamic outputs disaggregated into visual presentations and status states:

- JSP View Dispatching: Servlets forward validated data models to target JSP templates using RequestDispatcher.forward(request, response).
- Redirection & Flash Messages: On successful insert or delete operations, servlets issue response.sendRedirect() with status query parameters.
- Error Pages & Feedback Banners: Controlled exception handlers forward to styled error views displaying helpful diagnostic messages.

3.4 Integration / Interaction Interface

Component interaction follows a decoupled architectural pattern:

- Servlet-to-DAO Interaction: Controllers instantiate Data Access Objects (or invoke DBConnection) without hardcoding SQL in web layers.
- Database Factory (DBConnection.java): Centralized connection utility that inspects environment variables (MYSQLHOST, MYSQLPORT) and falls back to localhost JDBC URLs.
- Session Storage: Utilizes HttpSession to maintain logged-in user identifiers across subsequent page visits.

3.5 Non-Functional Internal Requirements

- Latency: Internal servlet execution and database query overhead must complete in under 150 milliseconds under standard load.
- Resource Reclamation: All Connection, PreparedStatement, and ResultSet objects must be explicitly closed in finally blocks or try-with-resources.
- Injection Prevention: 100% of SQL statements must use parameterized PreparedStatement syntax, eliminating SQL Injection vulnerabilities.
- Thread Safety: Servlets must not declare mutable instance variables, ensuring thread-safe concurrent execution across requests.

4. External Interface Requirements

4.1 User Interface

The user interface is designed with a modern, responsive layout that adapts seamlessly across desktop workstations, lab thin-clients, and mobile screens. Key interface screens include:

- Portal Shell & Navigation Bar: Universal header featuring the JavaLabPortal brand, breadcrumbs, role badges, and quick links.
- Unified Authentication Screen: Clean selectable login card for Students and Faculty with validation error indicators.
- Role Dashboards: Customized workspaces showing quick stats (total experiments, assigned sections, student count).
- Experiments Catalog Grid: Card-based visual directory displaying lab experiment numbers, titles, aim summaries, and action buttons.
- Management Data Tables: Tabular views for Students and Classes equipped with Add, Edit, and Delete action controls.
- Interactive Calculator Console: Visual keypad and expression display showing live results computed by the Java EE backend.

4.2 Software Interface

- Java Servlet Specification: Version 4.0 (javax.servlet and javax.servlet.http APIs).
- JavaServer Pages: Version 2.3 with Standard Tag Library (JSTL 1.2).
- Database Driver: MySQL Connector/J 8.0.33 JDBC Type-4 pure Java driver.
- Web Container: Apache Tomcat 9.0.x running on OpenJDK 11.

4.3 Hardware Interface

- Server Specifications: Standard x86_64 or ARM64 virtual server with minimum 1 vCPU, 512 MB RAM, and 1 GB available disk space.
- Client Specifications: Standard college desktop computer or laptop with minimum 1.0 GHz processor, 1 GB RAM, and a standard monitor (1024x768 resolution minimum).

4.4 Communication Interface

- Protocol: HTTP/1.1 and secure HTTPS (TLS 1.2/1.3 in production cloud deployment).
- Network Ports: Port 8080 (standard local Tomcat HTTP port) and Port 443 (Railway Cloud HTTPS edge).
- Data Exchange Format: Form-encoded HTTP parameters and semantic HTML5 DOM responses.

5. Other Functional Requirements

The functional requirements for JavaLabPortal are cataloged in the traceable matrix below. Each requirement represents a testable, discrete capability implemented within the Java EE architecture:

Requirement ID	Functional Requirement Description
FR-01	The system shall authenticate students and teachers against the MySQL database based on chosen role and credentials.
FR-02	The system shall invalidate active user sessions upon logout and redirect users securely to the login portal.
FR-03	The system shall permit authorized faculty to create, view, update, and remove student academic records.
FR-04	The system shall allow faculty instructors to organize, create, and list academic class sections and batch allocations.
FR-05	The system shall dynamically query and display the complete catalog of RTU Advance Java lab experiments.
FR-06	The system shall retrieve and render comprehensive theory notes, syntax references, and conceptual guides for each experiment.
FR-07	The system shall parse mathematical expressions submitted from the web client and evaluate computations via CalculatorServlet.
FR-08	The system shall accept two operands and an arithmetic operator to compute basic operations via ArithmeticServlet.
FR-09	The system shall allow students to submit star ratings and qualitative reviews for lab practicals via FeedbackServlet.
FR-10	The system shall dynamically inspect cloud environment variables (MYSQLHOST, MYSQLPORT) and establish resilient database connections.

6. Other Non-Functional Requirements

Non-functional requirements specify quality benchmarks, operational constraints, and performance parameters governing JavaLabPortal:

Quality Aspect	Non-Functional Specification & Measurable Benchmark
----------------	---

Performance	Page response and dynamic JSP rendering shall complete in under 500ms under normal classroom concurrency.
Reliability	The application shall maintain 99.5% operational availability throughout academic laboratory hours.
Scalability	The architecture shall easily scale horizontally across Tomcat container instances without state corruption.
Usability	The interface shall feature intuitive navigation requiring less than 5 minutes of onboarding for first-time students.
Maintainability	Code shall adhere to MVC separation of concerns, ensuring business logic changes do not affect JSP view templates.
Portability	The application shall package as a portable .war file executable on any compliant Servlet 4.0 / Tomcat 9 container.
Security	All database operations shall utilize parameterized queries; direct unauthorized access to management servlets shall be blocked.
Testability	Controllers, DAOs, and computational servlets shall be modular to facilitate JUnit testing and independent validation.
Efficiency	Database connection lifecycles shall be strictly managed to prevent connection pool exhaustion and memory leaks.

7. Analysis Models

7.1 Use Case Model

The Use Case model defines the functional interactions between external actors and the JavaLabPortal boundary:

- Primary Actors: Student, Faculty / Lab Instructor, System Administrator.
- Student Use Cases: Login, Browse Experiment Catalog, View Theory Content, Execute Interactive Calculator, Submit Lab Feedback, Logout.
- Faculty Use Cases: Login, Manage Students (Add/Edit/Delete), Manage Classes & Sections, View Experiment Catalog, View Student Reviews, Logout.
- System Boundary: Java EE Web Application containerized on Apache Tomcat 9, backed by MySQL 8.0.

7.2 Class / Module Architecture

The system's modular architecture is divided into three key tiers adhering to the Model-View-Controller design pattern:

- Controller Layer: LoginServlet, LogoutServlet, StudentServlet, ClassServlet, ExperimentServlet, TheoryServlet, CalculatorServlet, ArithmeticServlet, FeedbackServlet.
- Data Access & Utility Layer: DBConnection.java (Connection Factory), StudentDAO, ClassSectionDAO, ExperimentDAO, TheoryDAO, FeedbackDAO.

- Model / Entity Layer: Student (id, name, roll_no, email, class_id), Teacher (id, name, email, department), ClassSection (id, class_name, section, semester), Experiment (id, title, description, aim), TheoryContent (id, experiment_id, content), Feedback (id, student_id, experiment_id, rating, comments).
- View Layer: JSP templates incorporating JSTL tags and reusable CSS stylesheets located in WebContent/.

7.3 Activity / Workflow Diagram

The operational workflows of the portal follow structured activity sequences:

Student Practical Workflow:

1. Step 1: Student opens browser and navigates to the JavaLabPortal home URL.
2. Step 2: Student inputs university credentials and selects 'Student' role.
3. Step 3: LoginServlet validates credentials against student table and creates an active session.
4. Step 4: Student selects an assigned experiment from the Experiment Catalog.
5. Step 5: System retrieves detailed objectives, code templates, and theory manual.
6. Step 6: Student tests computational logic using interactive Calculator / Arithmetic tools.
7. Step 7: Student submits feedback ratings and logs out securely.

Faculty Management Workflow:

1. Step 1: Faculty logs in with instructor credentials.
2. Step 2: System renders the Faculty Management Dashboard.
3. Step 3: Faculty navigates to 'Manage Students' or 'Manage Classes'.
4. Step 4: Servlet processes add/update/delete operations and updates MySQL tables.
5. Step 5: System refreshes data view and displays success status banner.

8. Other Requirements

8.1 Data Residency and Database Persistence

All academic entity data (students, teachers, class sections, experiments, theory content, feedback) is persistently stored in a normalized MySQL 8.0 relational database. For local execution, data resides on the lab workstation's storage disk; in production, data is stored in Railway Cloud's managed persistent MySQL volume.

8.2 Privacy and Access Scopes

Data access is governed by role separation. Students have read-only access to curriculum experiments and theory content, and write-only access to their own feedback. Only authorized faculty and administrative accounts possess administrative privileges to modify class structures and student records.

8.3 Accessibility and Standards Compliance

The web user interface incorporates high-contrast color schemes, readable typography (Times New Roman and modern sans-serifs), and clear visual indicators for form validation. Interfaces conform to W3C HTML5 and CSS3 accessibility guidelines.

8.4 Deployment Environment

JavaLabPortal is engineered for dual deployment modes: (1) Local educational deployment on standalone Apache Tomcat 9 instances within college intranet networks, and (2) Cloud deployment via Dockerized containerization on Railway Cloud (live at <https://javalabportal-production.up.railway.app>), leveraging AWS Public ECR container base images.

8.5 Backup and Disaster Recovery

Database recovery is supported through SQL schema dumps (database.sql). Routine daily logical backups can be generated using standard mysqldump utilities, allowing restoration within 15 minutes in case of hardware or container failure.

8.6 Logging and Monitoring

Tomcat standard logging (catalina.out) records HTTP request access and servlet lifecycle events. Application-level error logging captures database connectivity failures and runtime exceptions, enabling swift diagnostic remediation.

8.7 System Dependencies

- Java Runtime Environment: OpenJDK 11 LTS / Java SE 8.
- Web Container: Apache Tomcat 9.0.x.
- Database Driver: mysql-connector-j-8.0.33.jar.
- Standard Tag Libraries: jstl-1.2.jar.

8.8 Training Materials & Documentation

A comprehensive README.md file is provided in the root project repository containing step-by-step installation instructions, Eclipse IDE setup workflows, database seeding procedures, and live cloud URLs to guide students and evaluators.